Seaborn Master

1. Environment Setup & Data Loading

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Inline magic to render plots directly below the code cell
%matplotlib inline

# Set a professional, consistent visual theme for the whole notebook
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.dpi'] = 100

print("Libraries imported successfully")
print("Seaborn version:", sns.__version__)
print("Pandas version:", pd.__version__)
```

```
Libraries imported successfully
Seaborn version: 0.13.2
Pandas version: 2.3.1
```

In [ ]:
```
```

2. Dataset Loading

In [2]:
```
# Load the built-in tips dataset directly from Seaborn's repository
df = sns.load_dataset("tips")

# Display the first 5 rows to verify the data loaded correctly
df.head()
```

Out[2]:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

In [ ]:
```
```

3. Data Understanding

3.1 df.head() — Peek at the First Rows

In [3]:
```
# View the top 5 rows of the loaded DataFrame
df.head()
```

Out[3]:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

3.2 df.info() — Structural Summary

In [4]:
```
# Inspect total counts, missing data flags, and column data types
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype   
---  ------      --------------  -----   
 0   total_bill  244 non-null    float64 
 1   tip         244 non-null    float64 
 2   sex         244 non-null    category
 3   smoker      244 non-null    category
 4   day         244 non-null    category
 5   time        244 non-null    category
 6   size        244 non-null    int64   
dtypes: category(4), float64(2), int64(1)
memory usage: 7.4 KB
```

3.3 df.shape — Dimensions

In [5]:
```
# Fetch total dimensions (rows, columns)
df.shape
```

Out[5]: (244, 7)

3.4 df.columns — Column Names

In [6]:
```
# List out all active column names
df.columns
```

Out[6]:
```
Index(['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size'], dtype='objec
t')
```

3.5 df.dtypes — Data Types Only

In [7]:
```
# Check isolated data type schemas
df.dtypes
```

Out[7]:
```
total_bill     float64
tip            float64
sex           category
smoker        category
day           category
time          category
size             int64
dtype: object
```

3.6 df.describe() — Statistical Summary (Numeric Columns)

In [8]:
```
# Generate structural statistics summary (mean, std, min, quartiles, max)
df.describe()
```

Out[8]:

	total_bill	tip	size
count	244.000000	244.000000	244.000000
mean	19.785943	2.998279	2.569672
std	8.902412	1.383638	0.951100
min	3.070000	1.000000	1.000000
25%	13.347500	2.000000	2.000000
50%	17.795000	2.900000	2.000000
75%	24.127500	3.562500	3.000000
max	50.810000	10.000000	6.000000

3.7 df.isnull().sum() — Missing Value Check

In [9]:
```
# Audit the dataset for null values
df.isnull().sum()
```

Out[9]:
```
total_bill    0
tip           0
sex           0
smoker        0
day           0
time          0
size          0
dtype: int64
```

3.8 df.duplicated().sum() — Duplicate Row Check

In [10]:
```
# Audit the dataset for duplicate records
df.duplicated().sum()
```

Out[10]:
```
1
```

3.9 .value_counts() — Frequency of Categories

In [11]:
```
# Analyze operational day distribution traffic volumes
df['day'].value_counts()
```

Out[11]:
```
day
Sat     87
Sun     76
Thur    62
Fri     19
Name: count, dtype: int64
```

3.10 .unique() — List Distinct Values

In [12]:
```
# List unique operational day labels
df['day'].unique()
```

Out[12]:
```
['Sun', 'Sat', 'Thur', 'Fri']
Categories (4, object): ['Thur', 'Fri', 'Sat', 'Sun']
```

3.11 .nunique() — Count of Distinct Values

In [13]:
```
# Loop through all columns and print unique value counts dynamically
for col in df.columns:
    print(f"{col:12s} -> {df[col].nunique()} unique values")
```

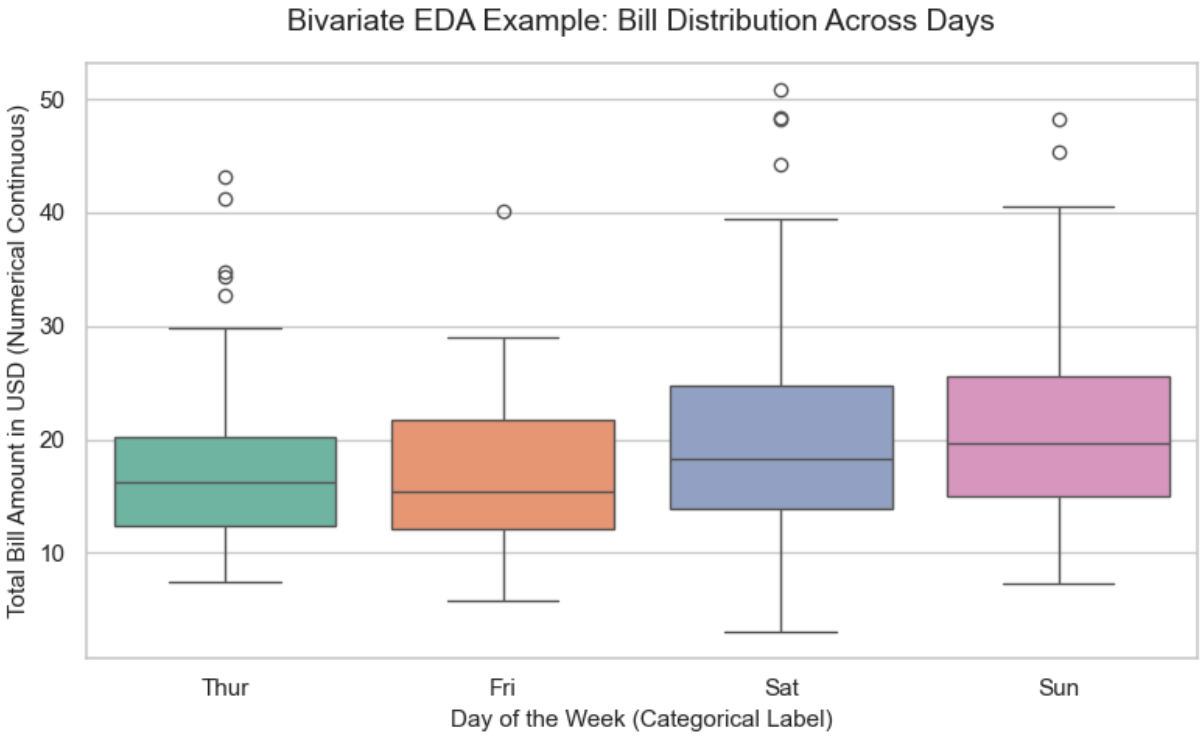
```
total_bill   -> 229 unique values
tip          -> 123 unique values
sex          -> 2 unique values
smoker       -> 2 unique values
day          -> 4 unique values
time         -> 2 unique values
size         -> 6 unique values
```

In [ ]:
```

```

4. EDA Concept - The Thoery Before The Chart

In [14]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Load the official 'tips' dataset (Zero manual typing = Zero syntax errors)
df = sns.load_dataset("tips")
```

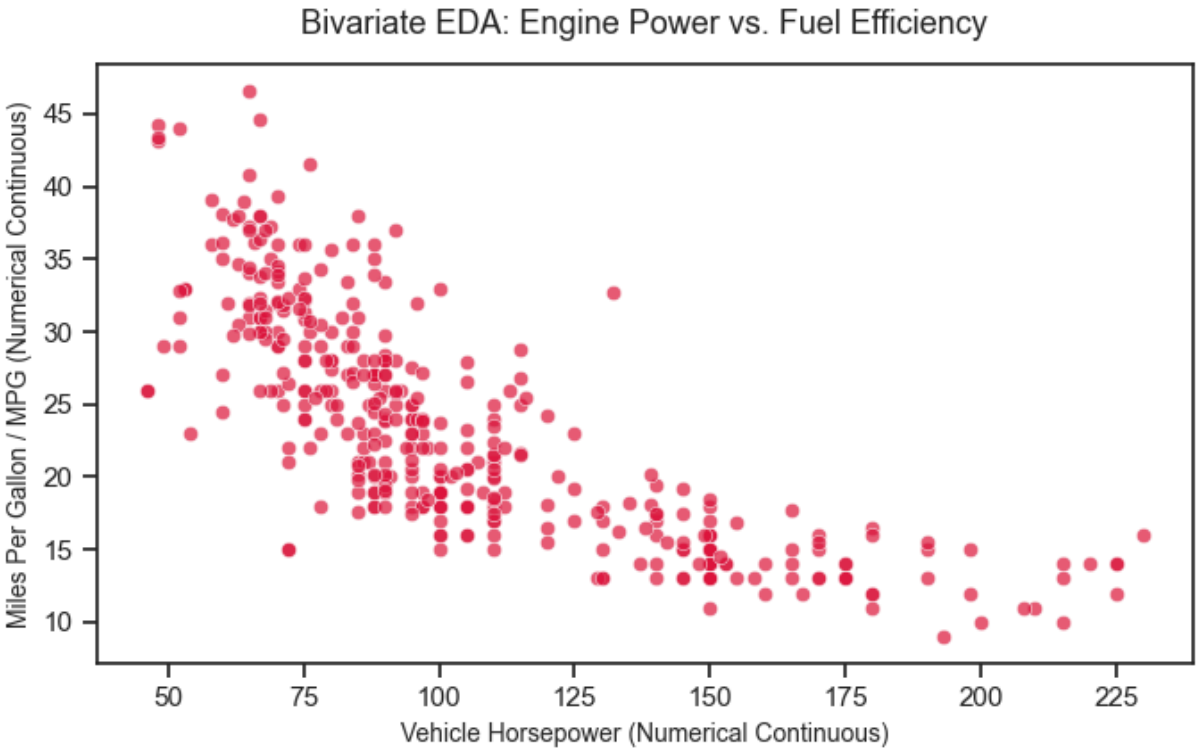
```
# 2. Set the global plotting theme
sns.set_theme(style="whitegrid")
plt.figure(figsize=(8, 5))

# 3. Bivariate Analysis Example: Categorical Variable vs. Numerical Variable
# 'day' is Categorical (Nominal), 'total_bill' is Numerical (Continuous)
sns.boxplot(data=df, x='day', y='total_bill', hue='day', palette='Set2', legend=Fal

# 4. Add clear visual markers and annotations
plt.title('Bivariate EDA Example: Bill Distribution Across Days', fontsize=14, pad=
plt.xlabel('Day of the Week (Categorical Label)', fontsize=11)
plt.ylabel('Total Bill Amount in USD (Numerical Continuous)', fontsize=11)

# Display the clean plot window
plt.tight_layout()
plt.show()
```



In [15]:

```
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Load the official 'mpg' (Miles Per Gallon) vehicle dataset
car_df = sns.load_dataset("mpg")

# 2. Configure canvas layout settings
sns.set_theme(style="ticks")
plt.figure(figsize=(7, 4.5))

# 3. Bivariate Analysis: Numerical Continuous vs. Numerical Continuous
# 'horsepower' and 'mpg' are both quantitative continuous variables
sns.scatterplot(data=car_df, x='horsepower', y='mpg', alpha=0.7, color='crimson')

# 4. Apply clean descriptive structural boundaries
```

```
plt.title('Bivariate EDA: Engine Power vs. Fuel Efficiency', fontsize=13, pad=12)
plt.xlabel('Vehicle Horsepower (Numerical Continuous)', fontsize=10)
plt.ylabel('Miles Per Gallon / MPG (Numerical Continuous)', fontsize=10)

# Render output window
plt.tight_layout()
plt.show()
```



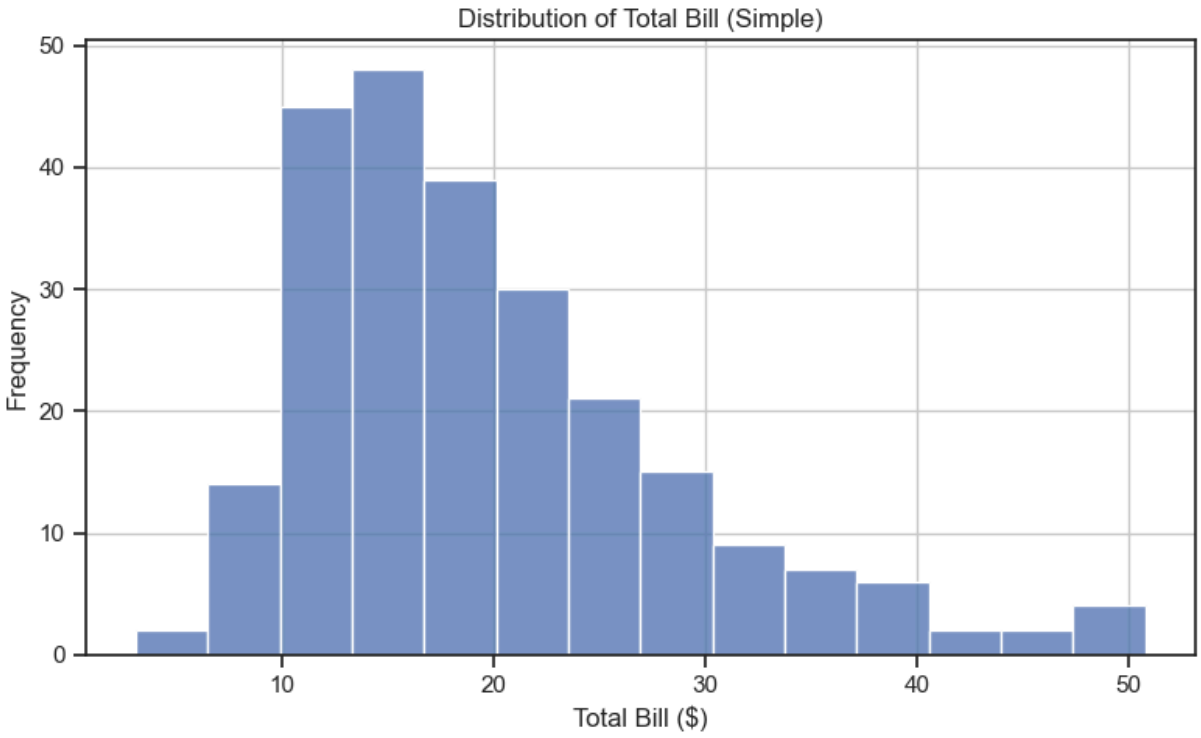
In [ ]:

5. Histogram - sns.histplot()/sns.displot

Notes pdf examples

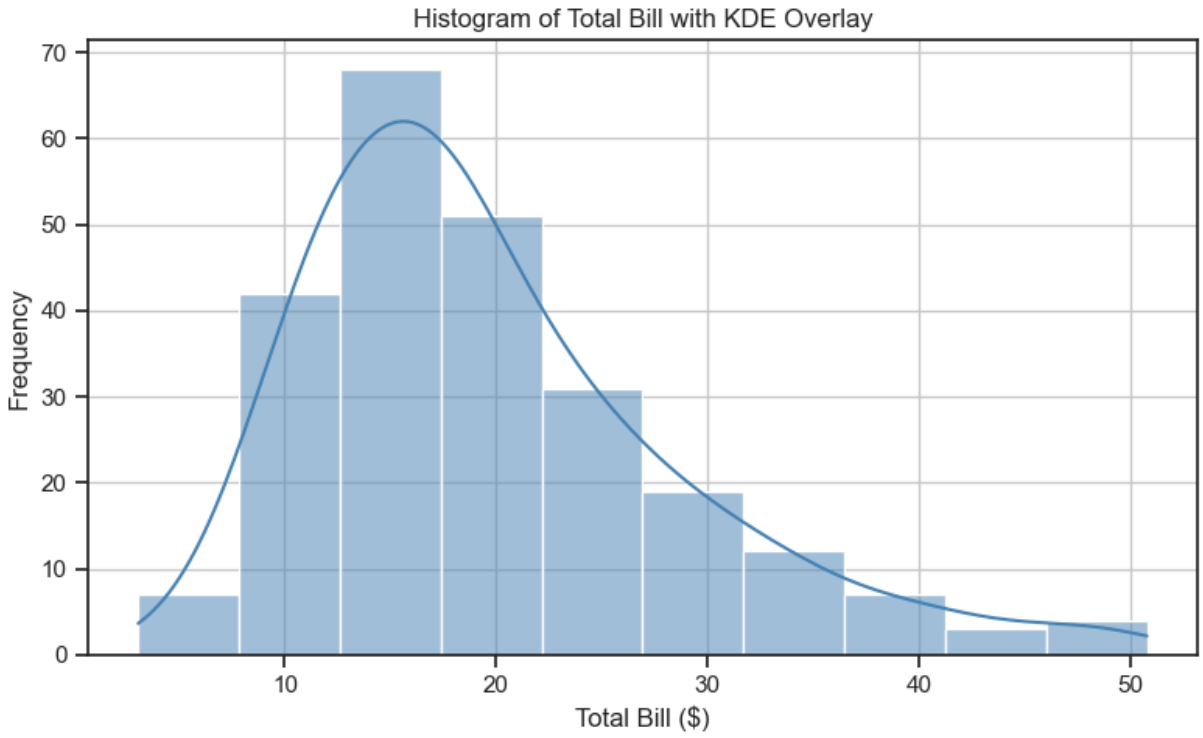
Simple Example

In [16]:
```
plt.figure(figsize=(8,5))
sns.histplot(df['total_bill'])
plt.title('Distribution of Total Bill (Simple)')
plt.xlabel('Total Bill ($)')
plt.ylabel('Frequency')
plt.grid(True)
plt.tight_layout()
plt.show()
```



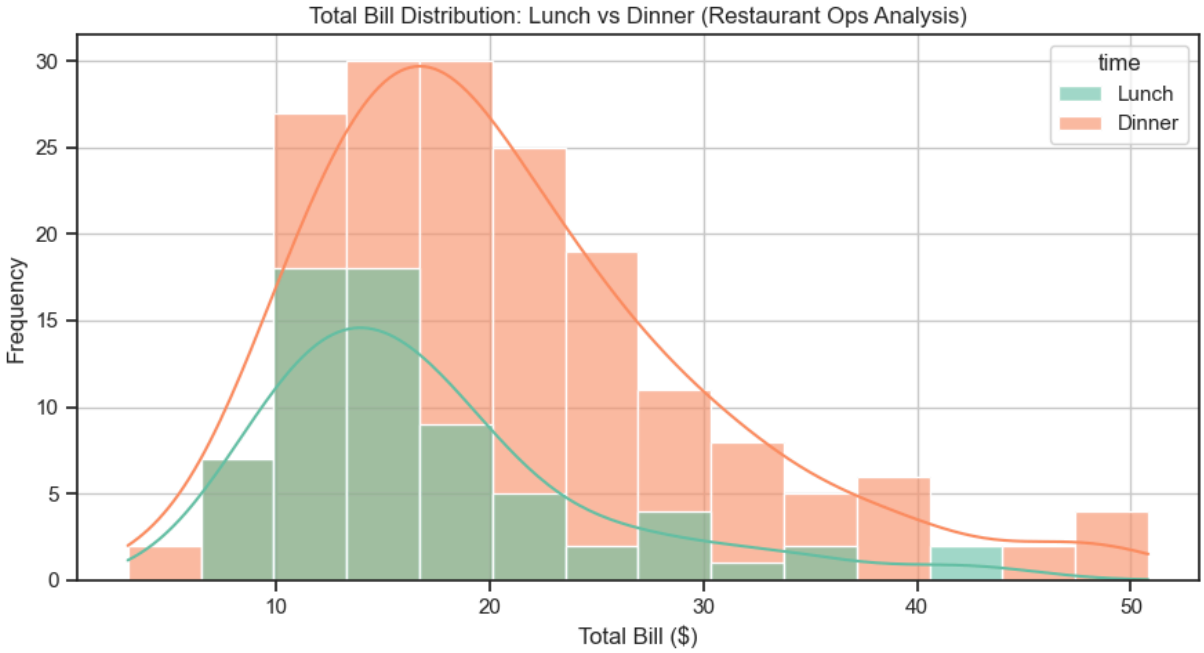
Intermediate Example — With Bins & KDE

In [17]:
```
plt.figure(figsize=(8,5))
sns.histplot(df['total_bill'], bins=10, kde=True, color='steelblue')
plt.title('Histogram of Total Bill with KDE Overlay')
plt.xlabel('Total Bill ($)')
plt.ylabel('Frequency')
plt.grid(True)
plt.tight_layout()
plt.show()
```



Real Industry Example — Restaurant Revenue Analysis

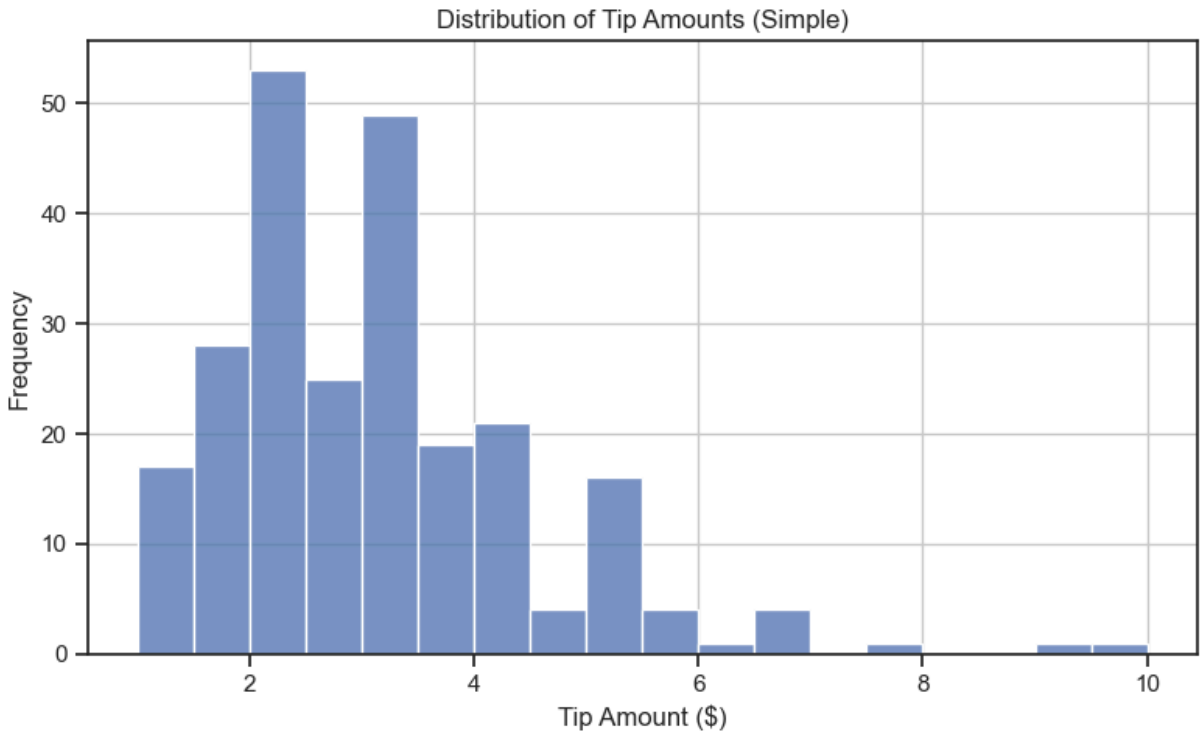
In [18]:
```
plt.figure(figsize=(9,5))
sns.histplot(data=df, x='total_bill', hue='time', kde=True,
             multiple='layer', palette='Set2', alpha=0.6)
plt.title('Total Bill Distribution: Lunch vs Dinner (Restaurant Ops Analysis)')
plt.xlabel('Total Bill ($)')
plt.ylabel('Frequency')
plt.grid(True)
plt.tight_layout()
plt.show()
```



Another Examples

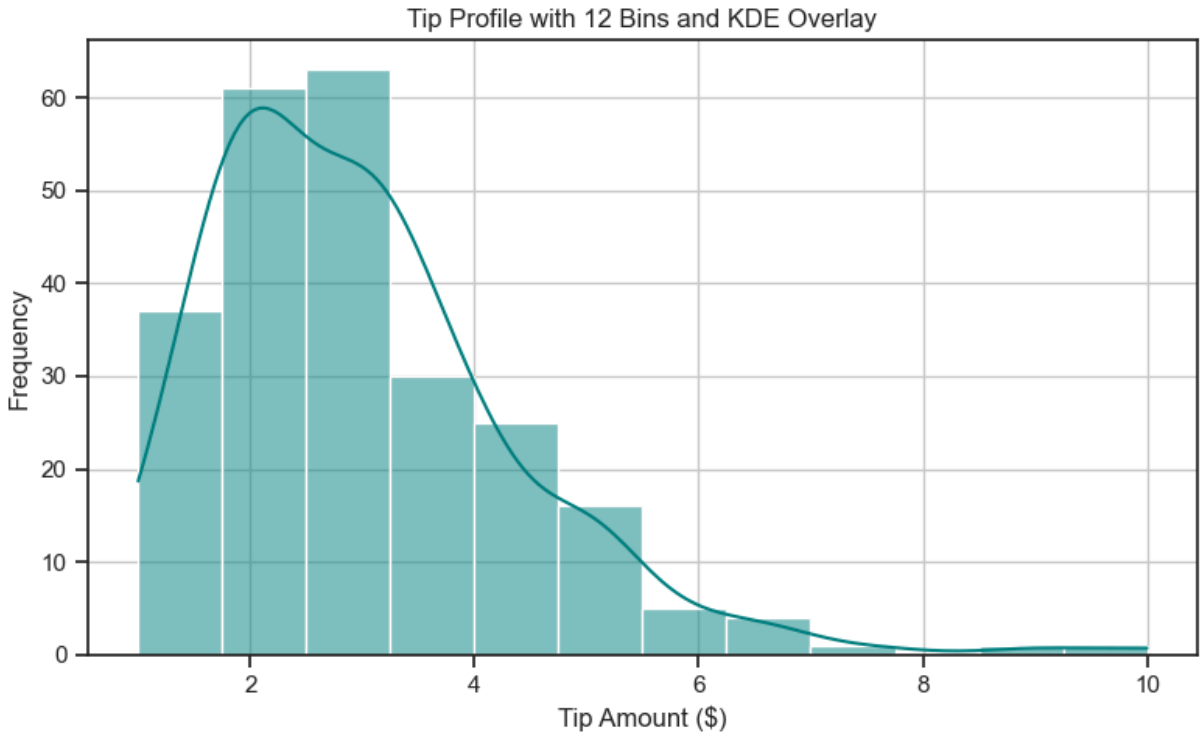
Simple Example (Applied to tip)

In [19]:
```
plt.figure(figsize=(8,5))
sns.histplot(df['tip'])
plt.title('Distribution of Tip Amounts (Simple)')
plt.xlabel('Tip Amount ($)')
plt.ylabel('Frequency')
plt.grid(True)
plt.tight_layout()
plt.show()
```



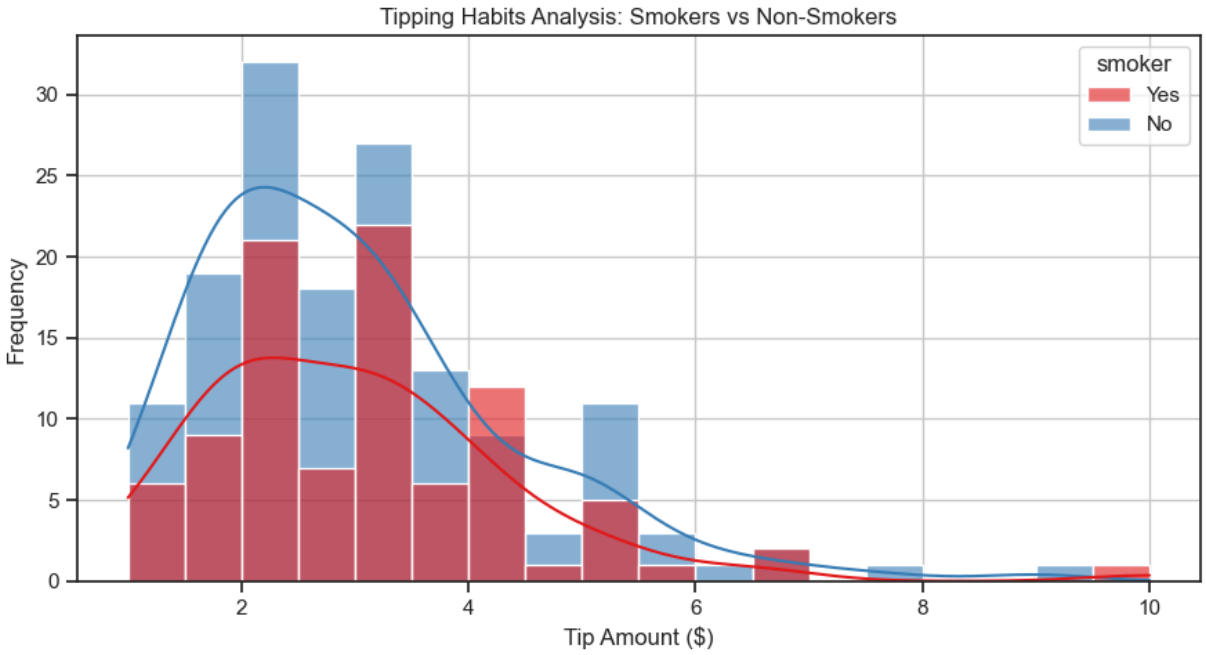
Intermediate Example — With Bins & KDE (Applied to tip)

In [20]:
```
plt.figure(figsize=(8,5))
sns.histplot(df['tip'], bins=12, kde=True, color='teal')
plt.title('Tip Profile with 12 Bins and KDE Overlay')
plt.xlabel('Tip Amount ($)')
plt.ylabel('Frequency')
plt.grid(True)
plt.tight_layout()
plt.show()
```



Real Industry Example — Customer Segment Behavior Analysis (Applied to tip)

In [21]:
```
plt.figure(figsize=(9,5))
sns.histplot(data=df, x='tip', hue='smoker', kde=True,
             multiple='layer', palette='Set1', alpha=0.6)
plt.title('Tipping Habits Analysis: Smokers vs Non-Smokers')
plt.xlabel('Tip Amount ($)')
plt.ylabel('Frequency')
plt.grid(True)
plt.tight_layout()
plt.show()
```

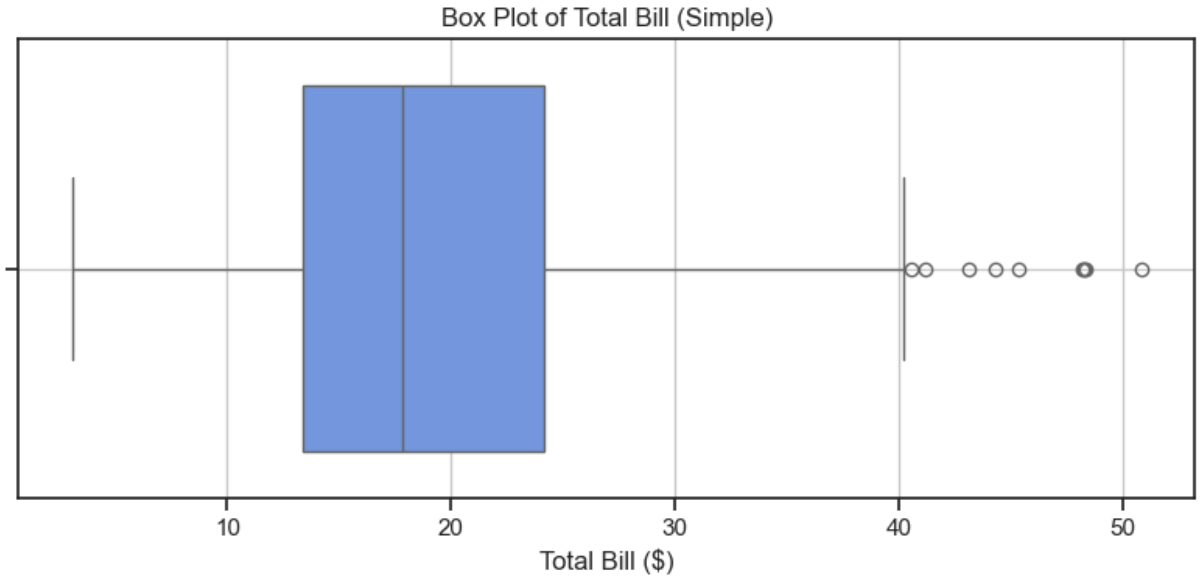


In [ ]:

6. Boxplot - sns.bosplot()

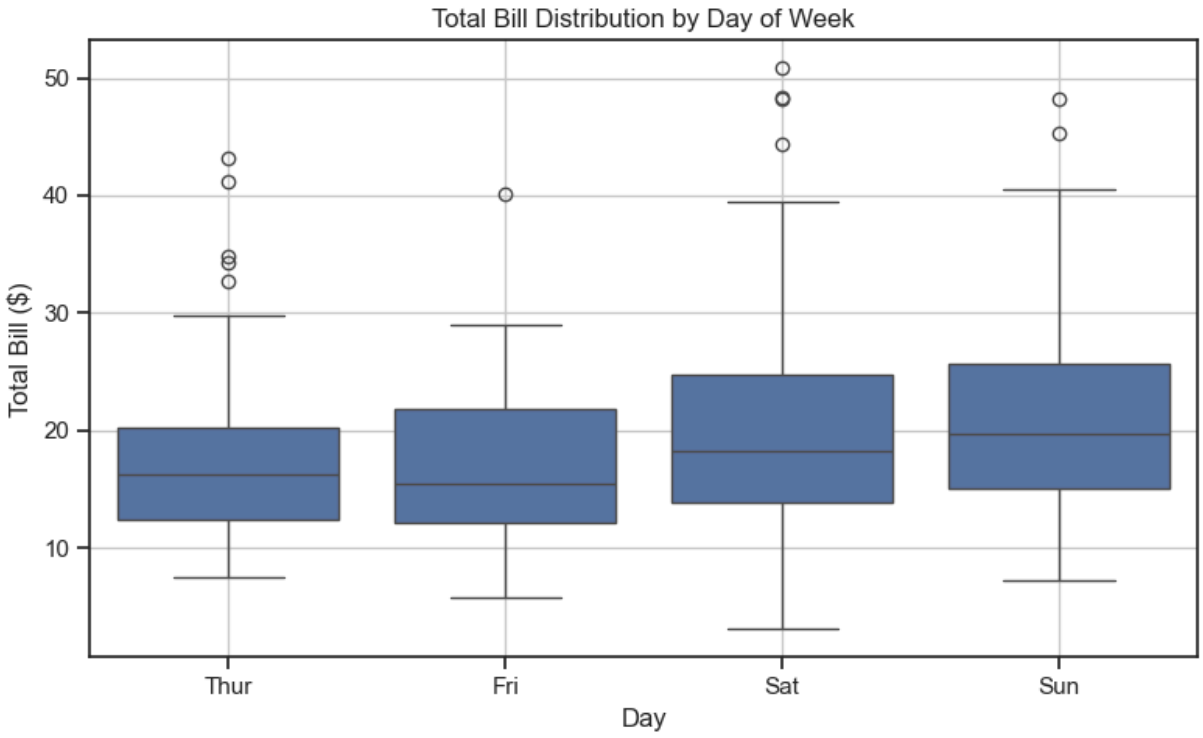
Simple Example

In [22]:
```
plt.figure(figsize=(8, 4))
sns.boxplot(x=df['total_bill'], color='cornflowerblue')
plt.title('Box Plot of Total Bill (Simple)')
plt.xlabel('Total Bill ($)')
plt.grid(True)
plt.tight_layout()
plt.show()
```



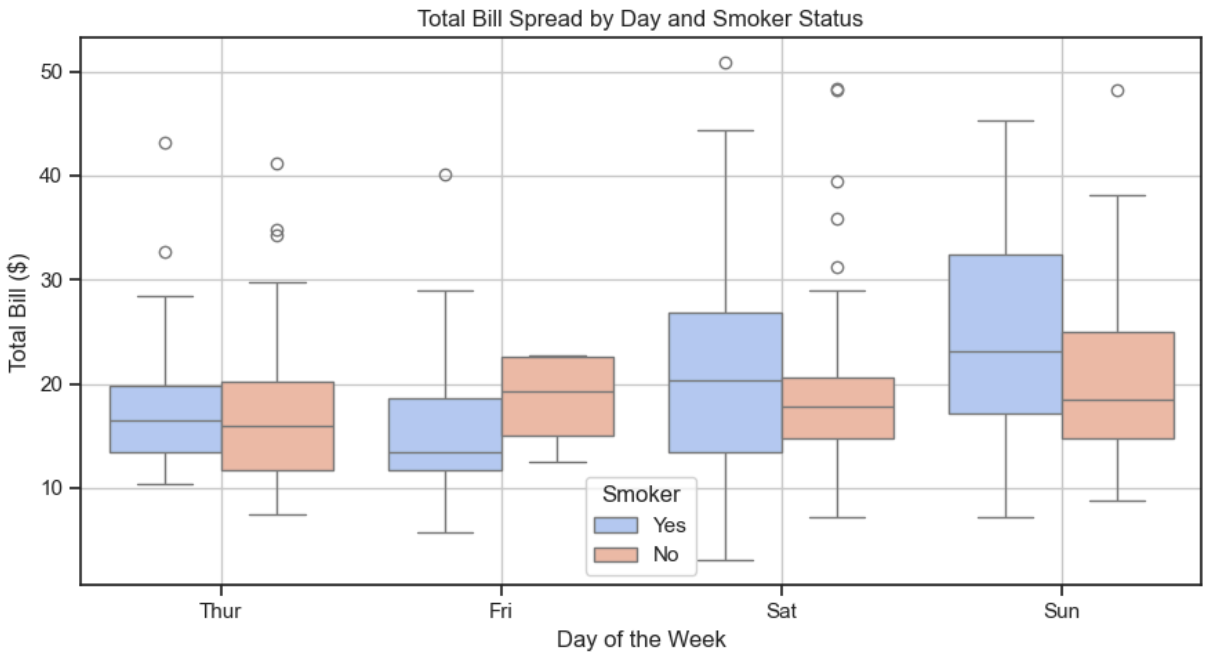
Intermediate Example Grouped by Day

In [24]:
```
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='day', y='total_bill')
plt.title('Total Bill Distribution by Day of Week')
plt.xlabel('Day')
plt.ylabel('Total Bill ($)')
plt.grid(True)
plt.tight_layout()
plt.show()
```



Real Industry Example: Grouped by Day and Smoker Status

In [25]:
```
plt.figure(figsize=(9, 5))
sns.boxplot(data=df, x='day', y='total_bill', hue='smoker', palette='coolwarm', dod
plt.title('Total Bill Spread by Day and Smoker Status')
plt.xlabel('Day of the Week')
plt.ylabel('Total Bill ($)')
plt.legend(title='Smoker')
plt.grid(True)
plt.tight_layout()
plt.show()
```

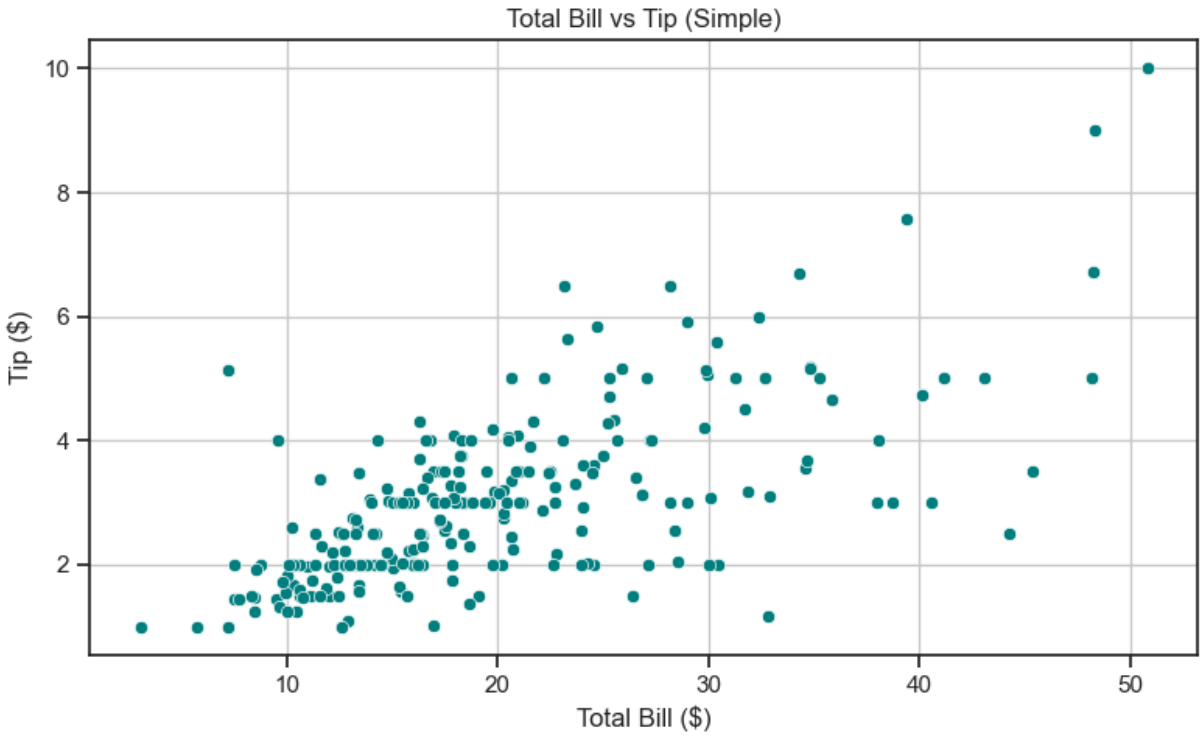


In [ ]:

7. Scatter Plot - sns.Scatterplot()

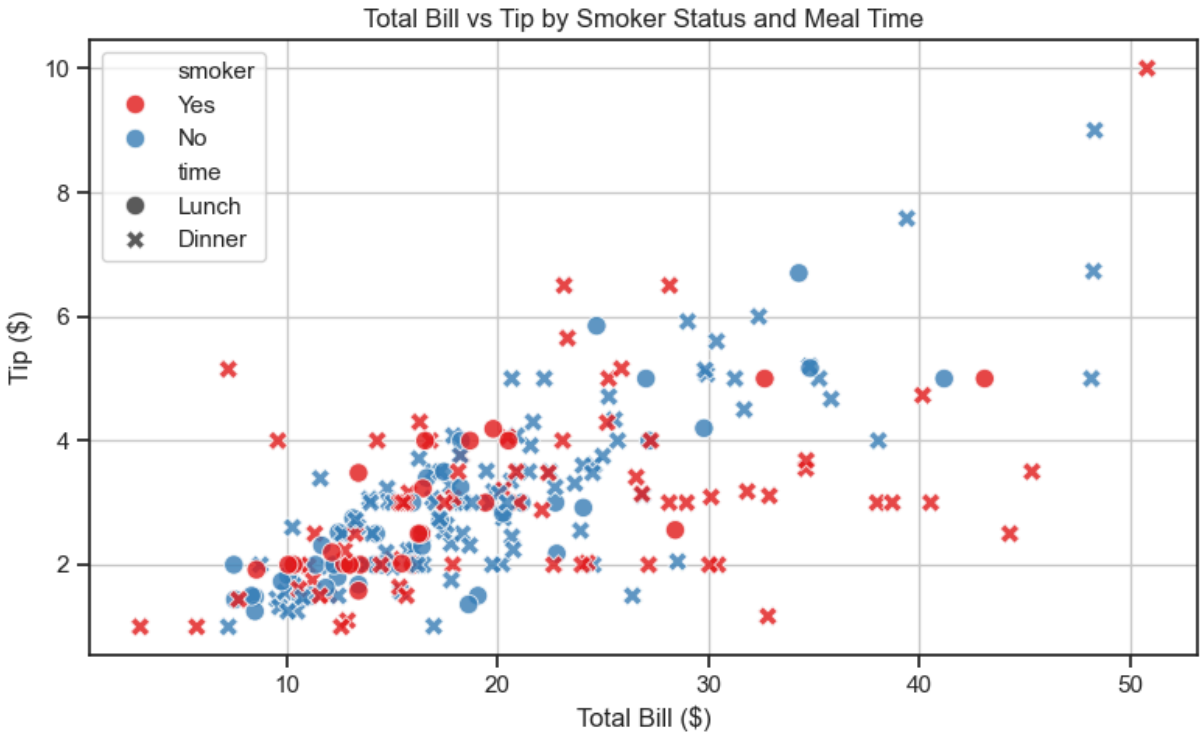
Simple Example

In [26]:
```
plt.figure(figsize=(8, 5))
sns.scatterplot(x=df['total_bill'], y=df['tip'], color='teal')
plt.title('Total Bill vs Tip (Simple)')
plt.xlabel('Total Bill ($)')
plt.ylabel('Tip ($)')
plt.grid(True)
plt.tight_layout()
plt.show()
```



Intermediate Example - With hue and style

In [27]:
```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='total_bill', y='tip', hue='smoker', style='time', palet
plt.title('Total Bill vs Tip by Smoker Status and Meal Time')
plt.xlabel('Total Bill ($)')
plt.ylabel('Tip ($)')
plt.grid(True)
plt.tight_layout()
plt.show()
```



Real Industry Example - Multivariate Analysis (E-commerce Reframing) ---

In [28]:
```
plt.figure(figsize=(9, 6))
sns.scatterplot(data=df, x='total_bill', y='tip', hue='day', size='size', sizes=(30
plt.title('Order Value vs Delivery Fee, Sized by Basket Size')
plt.xlabel('Order Value ($) [Total Bill]')
plt.ylabel('Delivery Fee Paid ($) [Tip]')
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left', borderaxespad=0)
plt.grid(True)
plt.tight_layout()
plt.show()
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: